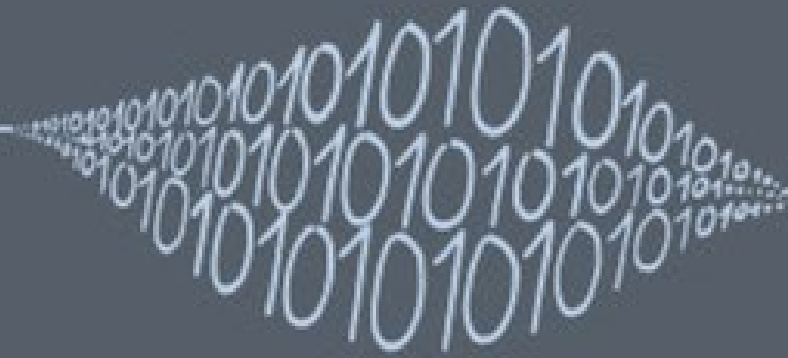
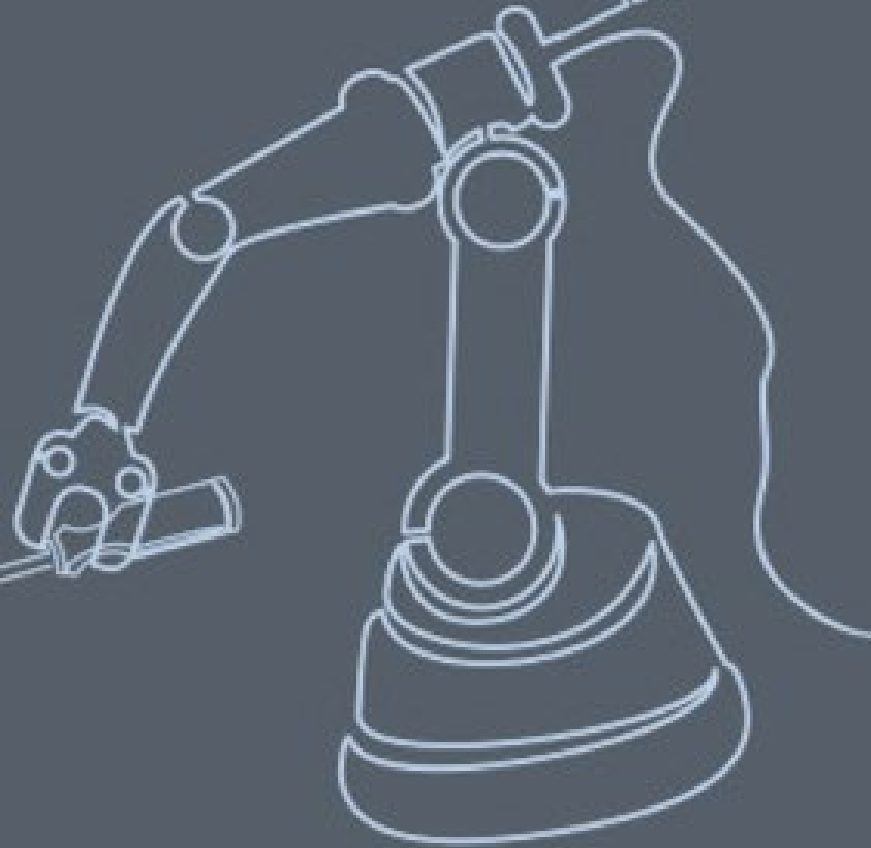


Einführung DZP

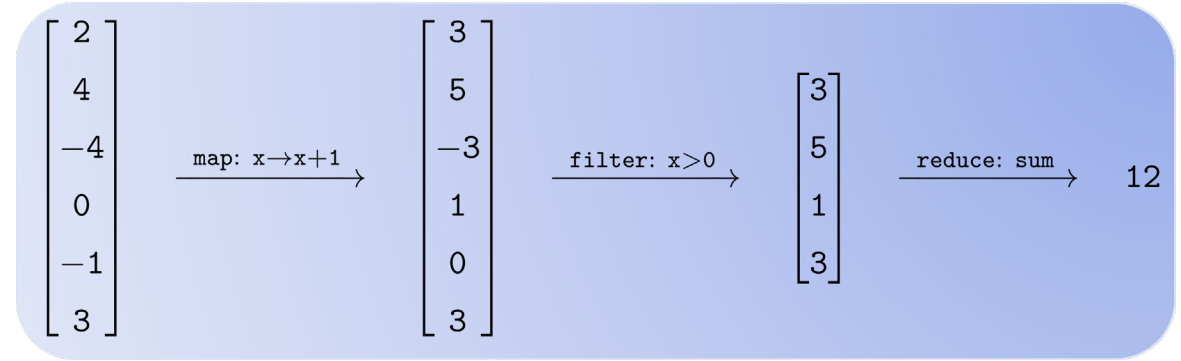
Datenzentriertes Programmieren

SW1



Inhalt

- **Einführung in das Modul**
 - Vorstellung
 - Lernziele
 - Einordnung in der Modullandschaft
 - Themenlandschaft / Semesterplan
 - Didaktisches Konzept / Arbeitsweise
 - Leistungsnachweis/Projektarbeit
 - Material und Arbeitsumgebung
 - Zielbild
- **Datenzentrierte Programmierung**
 - Grundprinzipien der Datenzentrierte Programmierung
- **Repetition List-Comprehensions (List/Dict)**
 - Live-Coding
- **Selbststudium**



Vorstellung: Melih Derman

- Studium
 - Lebensmitteltechnologie HSW/ZHAW (2004 – 2008)
 - MAS in Software Engineering HSR/OST (2019 – 2021)
- ZHAW seit 2008
 - Dozent Forschungsgruppe Simulation & Optimization
 - Lehre:
 - Programmieren in Python ADLS, BT, CH
 - Datenanalyse & Smart Processing LMs
 - Datenzentriertes Programmieren ADLS
- Sonst so
 - Familie: 2 Kinder
 - Hobbys: Fussball (Seniorenfussball, Juniorentainer und F-Junioren)



- derm@zhaw.ch
- www.zhaw.ch/=derm

Vorstellung: Lukas Hollenstein

- Theoretischer Physiker (Kosmologie)
 - Master Uni Zürich (2000 – 2005)
 - PhD Uni Portsmouth (2006 – 2009)
 - Postdoc Uni Genf & CEA Saclay (2009 – 2013)
- ZHAW seit 2013
 - Dozent Mathe & Simulation
 - Co-Leitung Forschungsschwerpunkt Digital Labs & Production
 - Leitung Forschungsgruppe Simulation & Optimization
- Sonst so
 - Familie: 2 Töchter
 - Musik: Schlagzeug, Jazz, Funk, Reggae
 - Life Long Learning

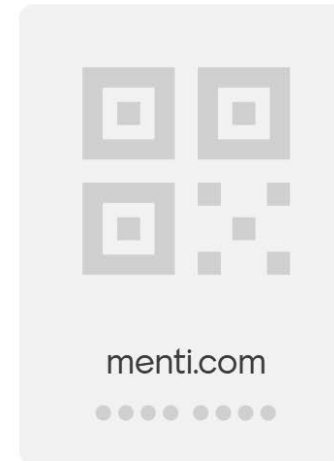


- hols@zhaw.ch
- www.zhaw.ch/=hols

Was versteht Ihr unter Datenzentriertes Programmieren?

Gehen Sie auf [menti.com](https://www.menti.com) | Code verwenden 8627 7928

Was versteht Du unter Datenzentriertes Programmieren?



Warten auf Teilnehmende



Menti

DZP-FS26-W1



Choose a slide to present

Was versteht Ihr unter Datenzentriertes Programmieren?

Goals of this course - reflection session

In this session, we will reflect on the goals of this course and what you need to do in order to reach them

Lernziele

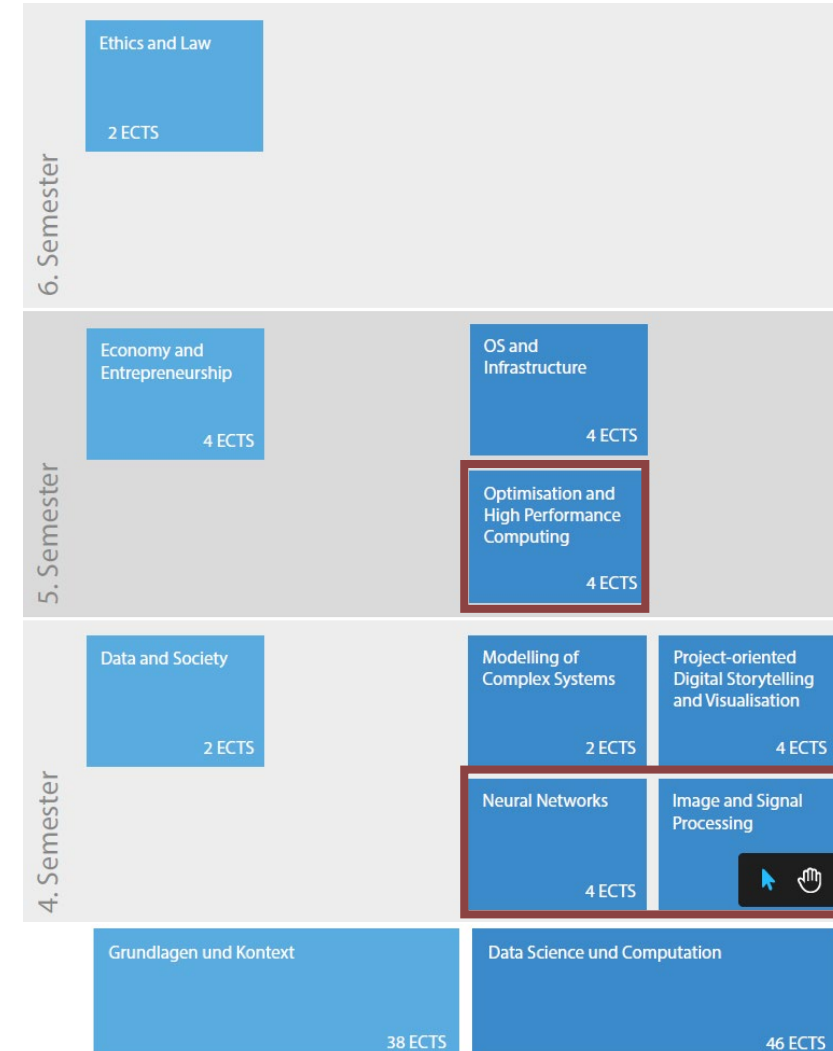
- Fachkompetenzen

- Mit kombinierten Datentypen umgehen und diese in Dataframes organisieren.
- Die wesentlichen Ansätze des funktionalen Programmierens kennen und mit Lambda-Funktionen arbeiten.
- Datentransformationen (Map), Auswahl (Filter) und einfache Aggregation (Reduce) auf Datensätzen durchführen.
- Programme zur Visualisierung strukturierter Daten erstellen.

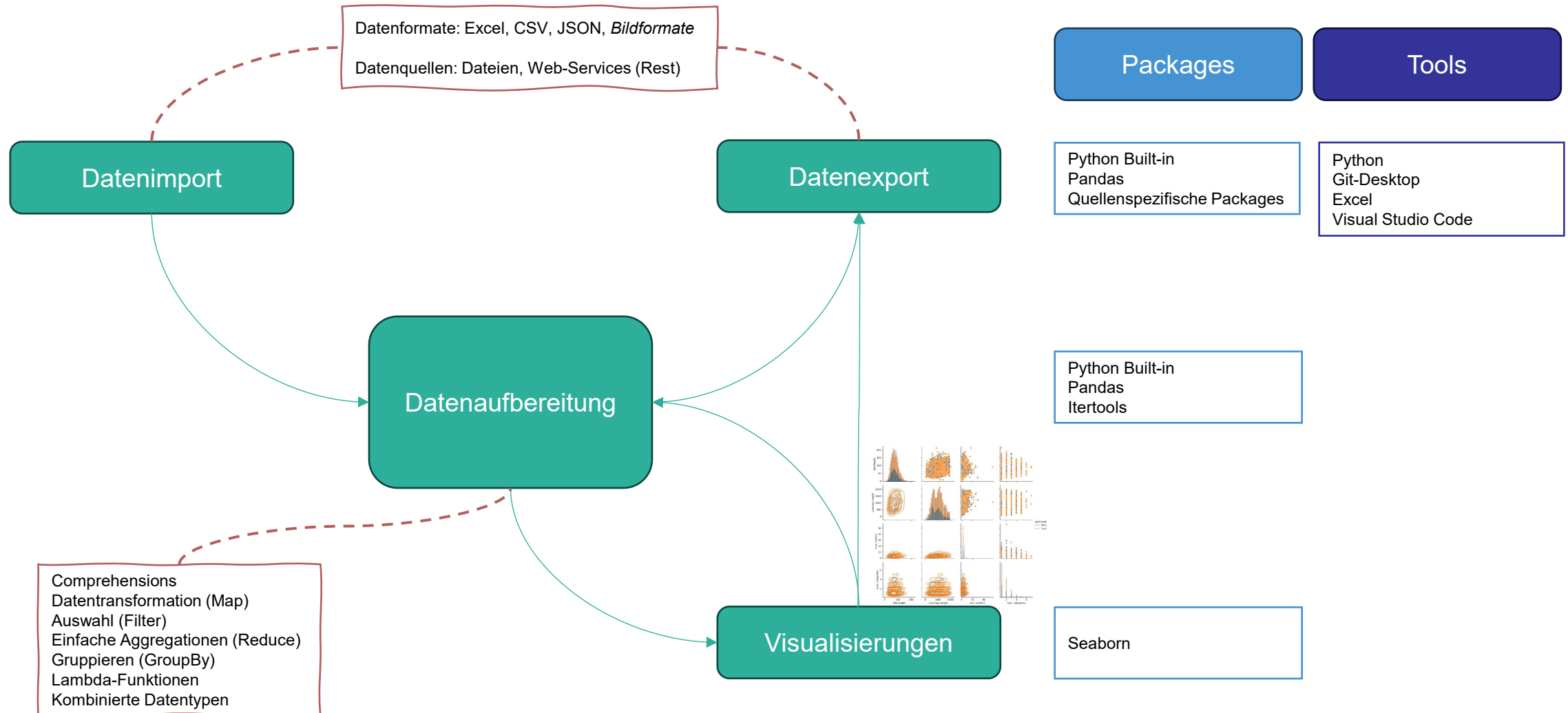
- Sachkompetenzen

- Python-Funktionen `map()`, `filter()` und `reduce()` als Alternativen zu Comprehensions einzusetzen.
- Package Pandas zu nutzen, um Daten einzulesen, auszuwählen, zu transformieren und häufige Statistiken darüber zu berechnen.
- Package Seaborn zu nutzen, um explorative Datenvisualisierungen zu erstellen.
- Sich beim Programmieren anhand von Dokumentation und Online-Ressourcen selbst helfen
- Über Arbeitsweise von Programmcode reflektieren

Abhängigkeiten zwischen den Modulen



Themenlandschaft



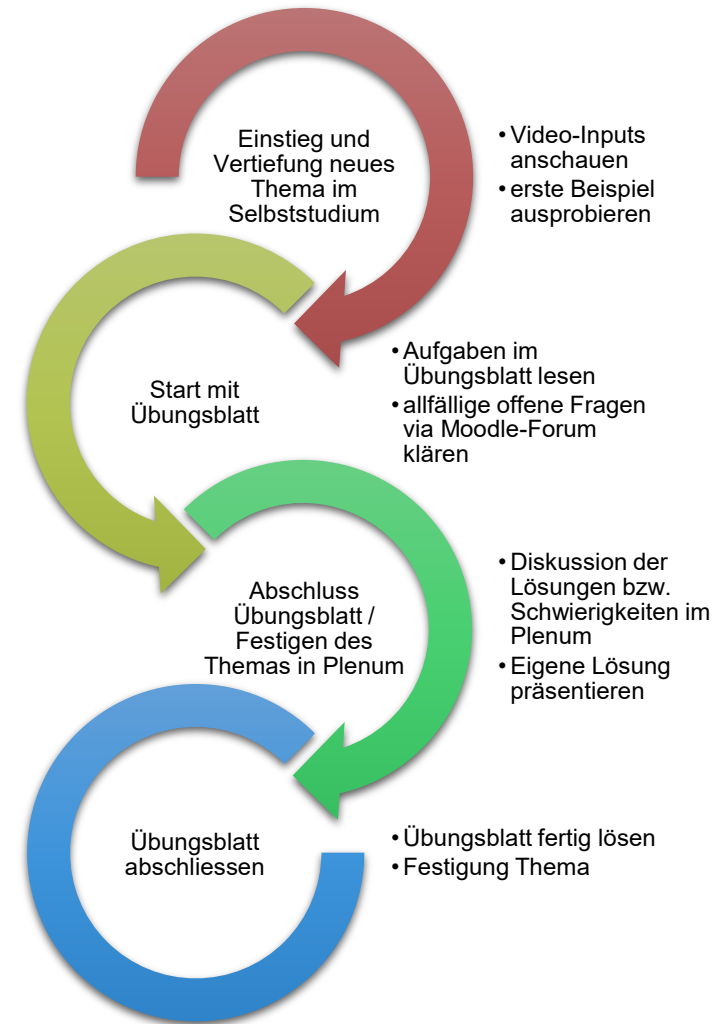
Semesterplan

Semesterplan: DZP AD25 FS26 (Stand: 17.02.2026)						
SW	KW	Tag	Datum	Thema	Ort	Info
1	8	Mi	18. Feb	Einführung DZP	GA 207	
		Fr	20. Feb	Repetition List-Comprehension	RX E0.03	
2	9	Mi	25. Feb		GA 207	
		Fr	27. Feb	Datenimport csv / map(), filter(), reduce() / lambda-Funktionen	online	asynchron
3	10	Mi	04. Mär		GA 207	
		Fr	06. Mär	Pandas-Basics (Series, Dataframe, Dataimport, Datenexport)	online	asynchron
4	11	Mi	11. Mär		GA 207	
		Fr	13. Mär	Konzept Datenrepräsentation/ Datenimport json / Datenquelle Web-API	online	asynchron
5	12	Mi	18. Mär		GA 207	
		Fr	20. Mär	Pandas - map, filter, apply	online	asynchron
6	13	Mi	25. Mär		GA 207	
		Fr	27. Mär	Seaborn - Catplot/Relplot/Heatmap	online	asynchron
7	14	Mi	01. Apr		GA 207	
		Fr	03. Apr	Pandas & Reduce / Groupby, Agg	online	Karfreitag/asynchron
8	15	Mi	08. Apr		GA 207	Projekt-Start
		Fr	10. Apr	Best Practices & Code Refactoring	online	asynchron
9	16	Mi	15. Apr	Leistungsnachweis / Recap: Beispiele zu Grundprinzipien	GA 207	
		Fr	17. Apr	Pandas: loop vs apply vs vektorisiert bzgl Speed	online	asynchron
10	17	Mi	22. Apr	Gruppenarbeit	GA 207	
		Fr	24. Apr	Datenexport/Repetition bzw. Gruppenarbeit	online	asynchron
11	18	Mi	29. Apr			Kulturtag
		Fr	01. Mai			Kulturtag/asynchron
12	19	Mi	06. Mai	Gruppenarbeit	GA 207	
		Fr	08. Mai	Gruppenarbeit	online	asynchron
13	20	Mi	13. Mai	Gruppenarbeit	GA 207	
		Fr	15. Mai		online	Brücke
14	21	Di	21. Mai		online	Projekt-Abgabe (23:30 Uhr)
		Mi	22. Mai	Präsentationen	GA 207	
		Fr	22. Mai	Präsentationen	online	
15	22	Mi	28. Mai	Präsentationen	GA 207	Abgabe Peer-Reviews (23:30 Uhr)
		Fr	30. Mai	Präsentationen - Puffer/Abschluss	online	

Didaktisches Konzept

- Ziele / Überzeugung
- Motivation durch konkrete Beispiele
 - Mehr Praxis – weniger Theorie
- Lernen durch aktives Tun
 - Viel selbstständige Arbeit
- Produktives Scheitern
 - Selbst ausprobieren – in der Gruppe diskutieren
 - Iterativ verbessern und vertiefen
- Flipped Classroom
 - Wissensvermittlung im Selbststudium mithilfe eines Arbeitsblatts & Videos
 - Vertiefung & Festigung

Arbeitsweise



Zeitlicher Ablauf

Freitag 08:50 bis 09:35 Uhr
(asynchron)

Vor Mittwoch

Mittwoch 13:50 bis 14:35 Uhr
(vor Ort)

Vor Freitag

Lernzyklus

Phase	Ziele	Aktivitäten	Sozialform	Medien / Material	Dauer / Aufwand	Zeitlicher Ablauf
Einstieg und Vertiefung neues Thema (bis Mi bzw. am Fr)	- Kennenlernen der Problemstellung & Methode - Erste Beispiele erleben - Einarbeitung & Vertiefung in die Thematik	- Video-Inputs anschauen - Erste Beispiel ausprobieren - Übung mit weiteren Themenbeispielen - Erklärung des Übungsblatts	Einzeln oder in Lerngruppen (frei)	Arbeitsblatt, Videos, Websites, Texte, Jupyter Notebooks	ca. 45 Min	Freitag 08:50 bis 09:35 Uhr (asynchron)
Start mit Übungsblatt (bis Mi)	- Aufgaben im Übungsblatt verstehen	- Aufgaben im Übungsblatt lesen - allfällige offene Fragen via Moodle-Forum klären - Fragen zum Thema/Übungen notieren für den Unterricht vor Ort - Mit ersten Aufgaben starten	Einzeln oder in Lerngruppen (frei)	Übungsblatt mit ca. 3 Aufgaben	ca. 30 Min	Vor Mittwoch
Abschluss Übungsblatt / Festigen des Themas in Plenum (Mi)	- Festigung Übungsblatt	- Übungsblatt abschliessen - Diskussion der Lösungen bzw. Schwierigkeiten im Plenum - Eigene Lösung präsentieren	Plenum (Think-Pair-Share)	Übungsblatt mit ca. 3 Aufgaben	45 Min	Mittwoch 13:50 bis 14:35 Uhr (vor Ort)
Übungsblatt abschliessen (bis Mi)	- Festigung Thema	- Übungsblatt fertig lösen - Allfällige offene Fragen via Moodle-Forum klären	Einzeln oder in Lerngruppen (frei)	Übungsblatt mit ca. 3 Aufgaben	ca. 15 Min	Vor Freitag
Projektarbeit in Gruppen	- Festigung der Themen - Anwenden der gelernten Methoden - Arbeit in Gruppe		in Gruppen			Selbstorganisati on

Leistungsnachweis

- **Keine** abgesetzte Modulprüfung
- Erfahrungsnote durch Gruppenarbeit (Projekt) & Moodle-Test
- Gruppenarbeit
 - Start ca. in Woche SW 8
 - Form: Jupiter-Notebook (Kombination zwischen Code und Erklärung)
 - Präsentation der Gruppenarbeit
 - **Gewichtung 70 %**
 - Details folgen später
- Moodle-Test
 - Datum: 15. April 2026
 - Form: Moodle-Test
 - **Gewichtung 30 %**
 - Details folgen später

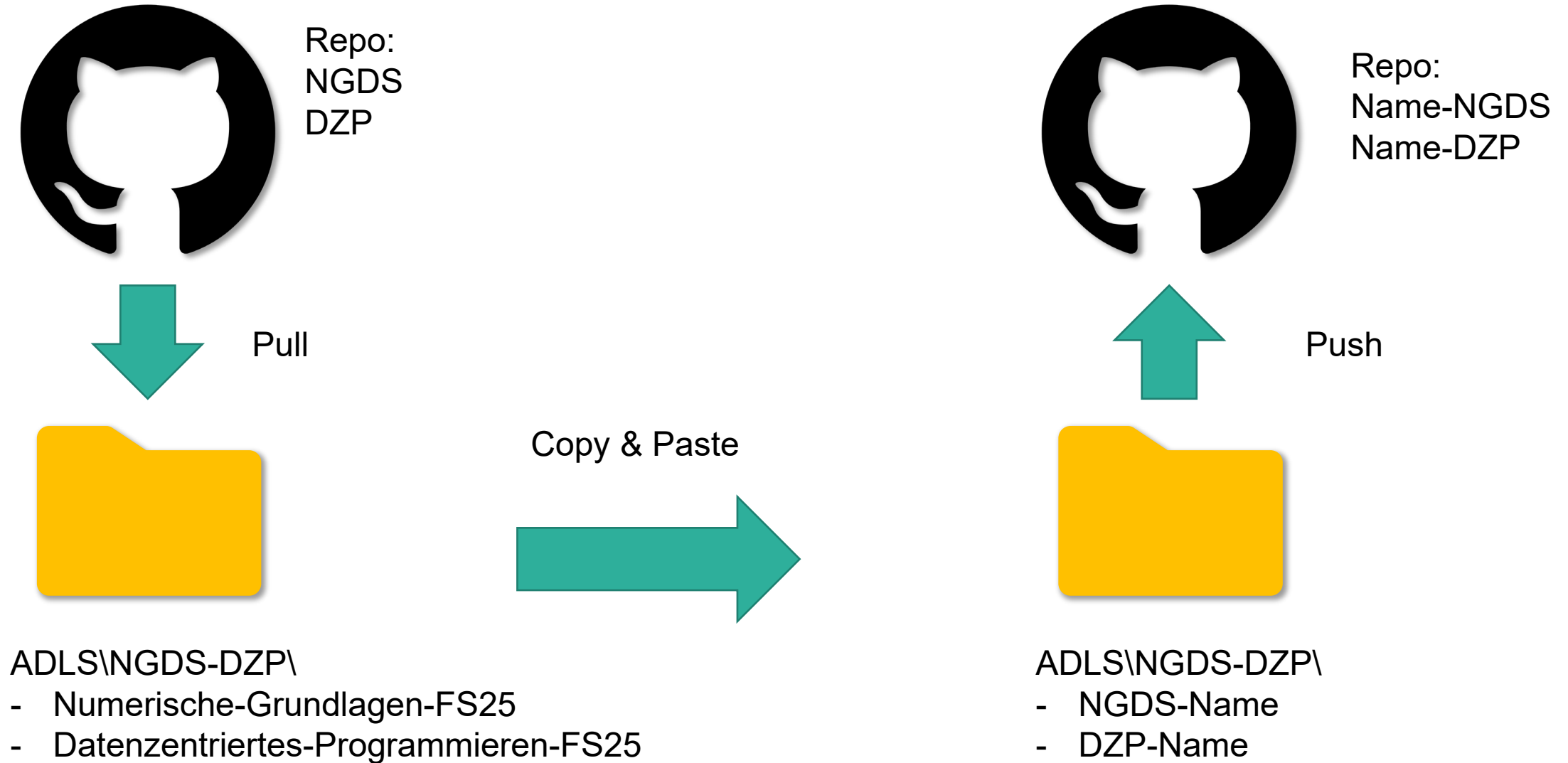
Material und Arbeitsumgebung

- [Moodle-Kurs](#) anschauen
 - Nachrichtenforum
 - Wichtige Dokumente
 - Links zu Ressourcen
- [GitHub Repo](#)
 - Anleitung
 - Arbeitsmaterial

Arbeitsumgebung aufsetzen

- Arbeitsverzeichnis erstellen
- GitHub Repository klonen
- Virtual Environment erzeugen
- (optional) Eigenes Repo aufsetzen

Arbeitsflow

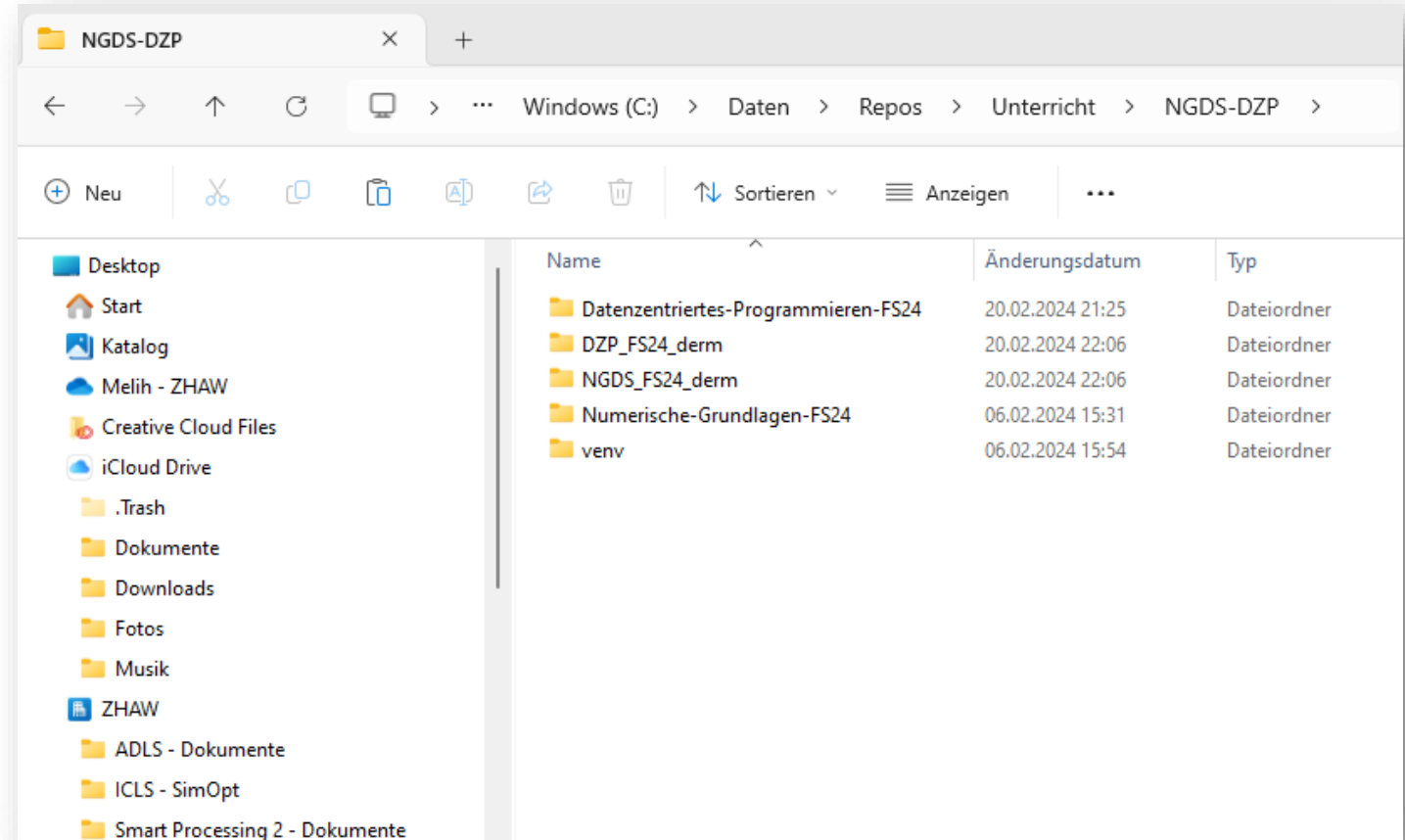


Lokale Ordner



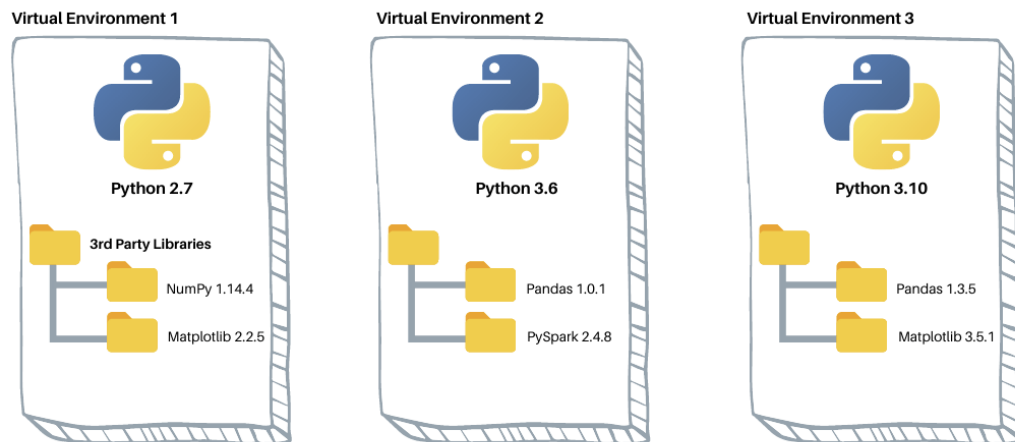
ADLS\NGDS-DZP\

- Numerische-Grundlagen-FS26
- Datenzentriertes-Programmieren-FS26
- NGDS-Name
- DZP-Name
- venv



Virtuelle Umgebungen (venv) in Python

- Eine virtuelle Umgebung (venv) ist ein isolierter Bereich für ein Python-Projekt. Sie sorgt dafür, dass alle installierten Pakete nur innerhalb dieses Projekts verwendet werden, ohne andere Projekte oder die Systeminstallation zu beeinflussen.



dataquest.io

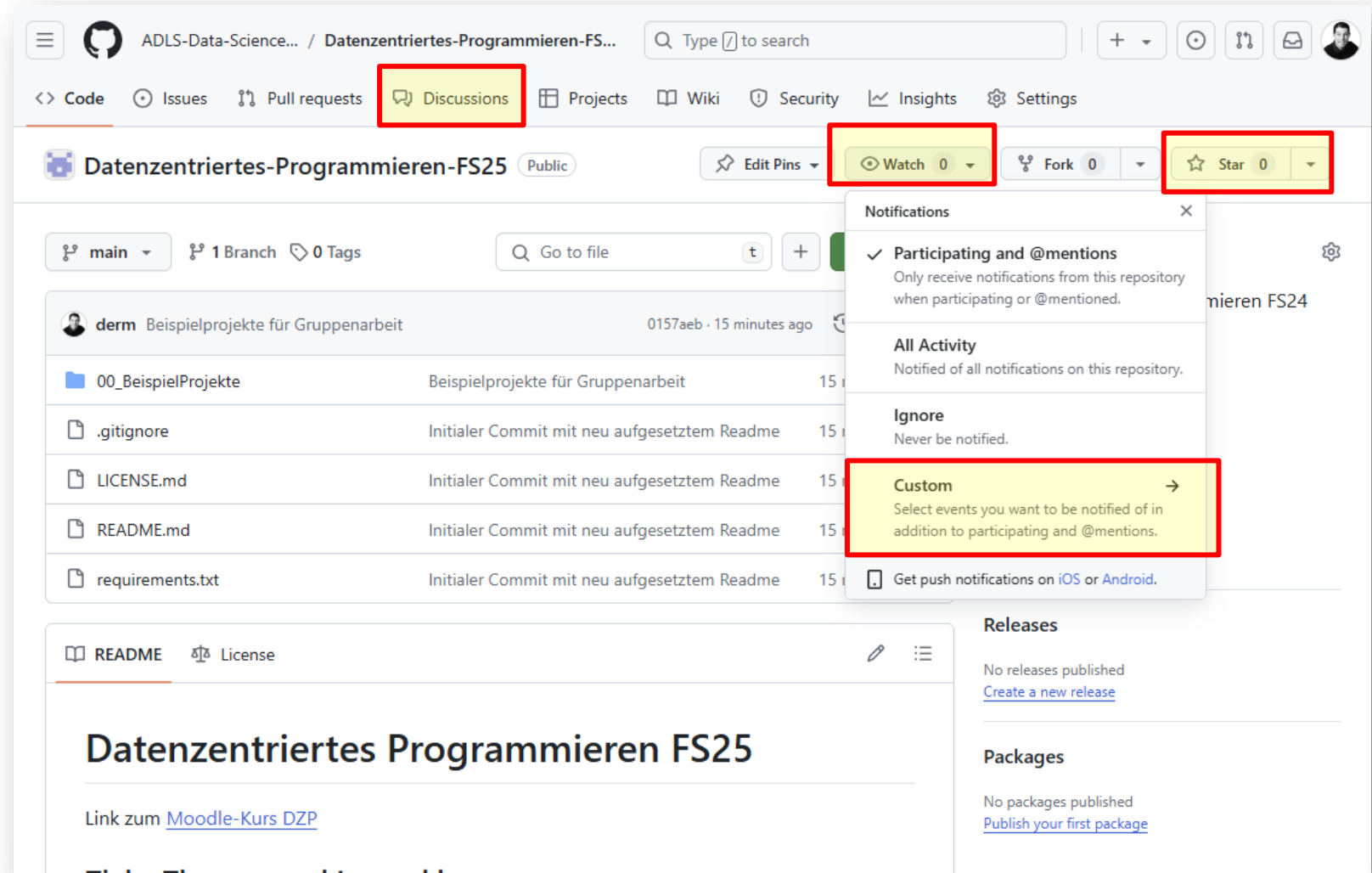
- **Vorteile**

- ✓ Jedes Projekt kann eigene Paket-Versionen nutzen.
- ✓ Keine Konflikte mit anderen Projekten oder globalen Installationen.
- ✓ Einfaches Teilen von Abhängigkeiten mit requirements.txt.
- ✓ Sicheres Testen neuer Bibliotheken.

Anleitung auf GitHub: [GitHub Repo](#)

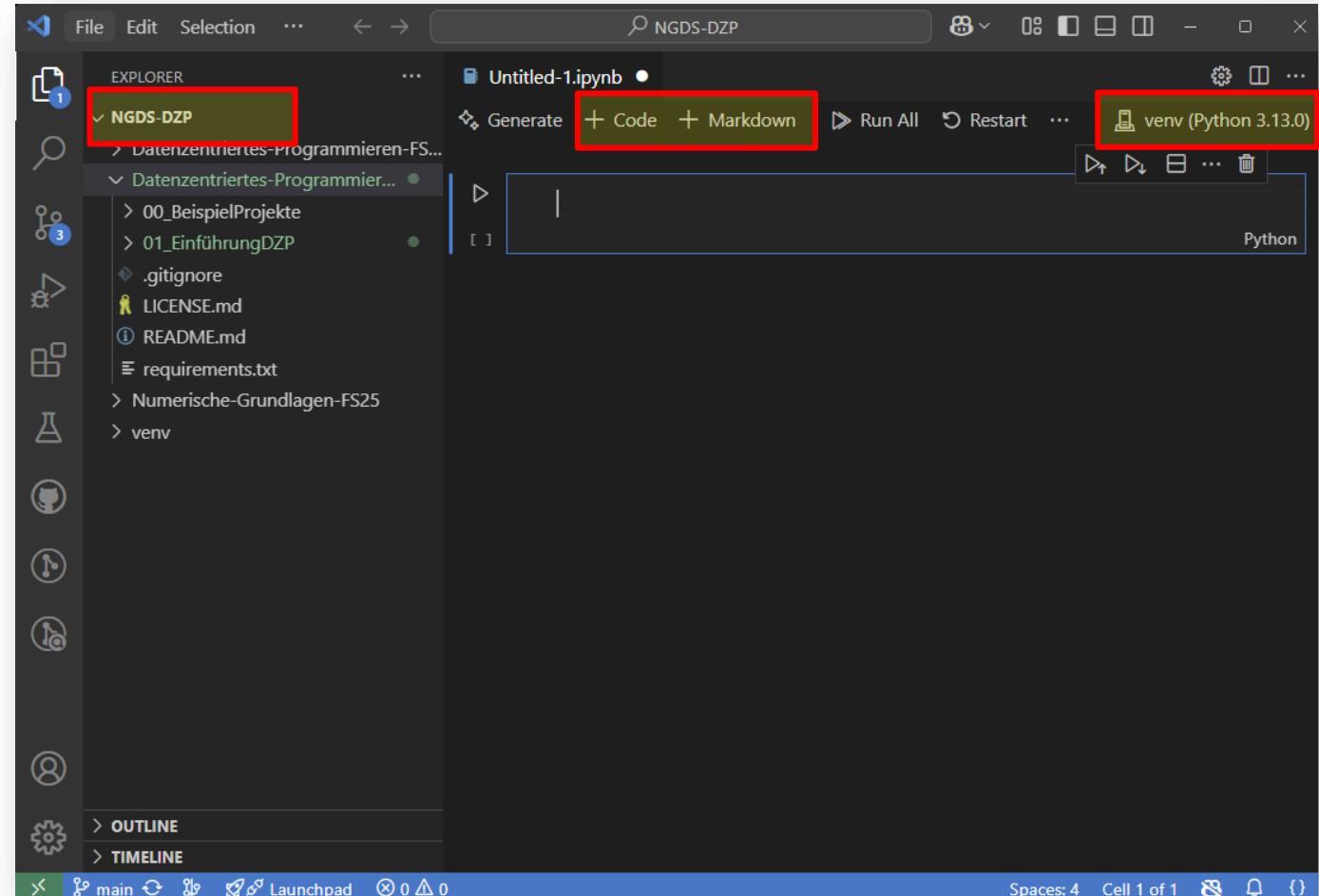
Fragen, Antworten und Feedback

- GitHub Repository
 - **Star**
→ schnell finden
 - **Watch**
→ Custom
→ Discussions
- Diskussionsforum
 - **Q&A:**
Fragen stellen und Antworten geben
 - **Ideas:**
Vorschläge und Feedback



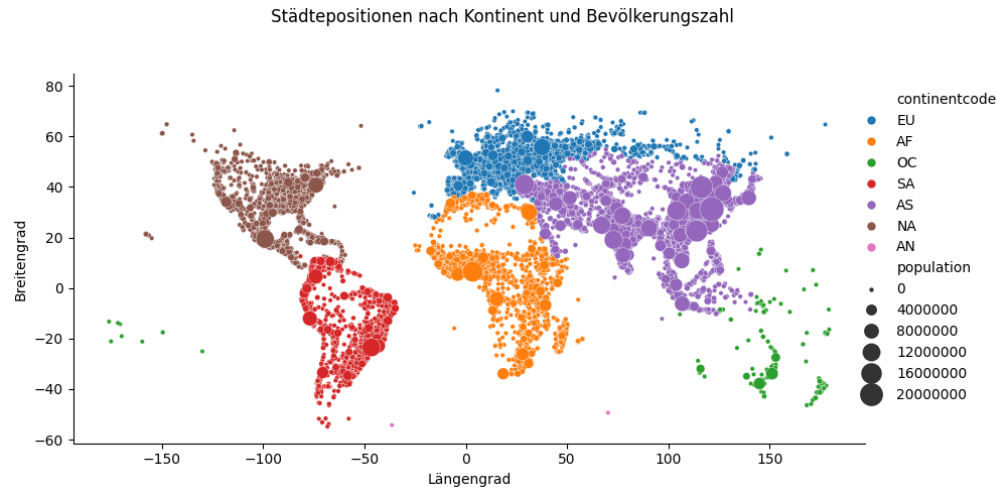
Arbeit mit Jupyter Notebooks

- Environment korrekt?
- Zellen
 - Code
 - Markdown
- Ausführen
 - Ctrl + Enter: ausführen
 - Shift + Enter: ausführen & weiter
 - Alt + Enter: ausführen & neue Zelle
- Markdown (Text)
 - ****fett****, *__kursiv__*
 - $\$math\$$ (LaTeX)
 - # Titel 1
 - ## Titel 2
 - ### Titel 3

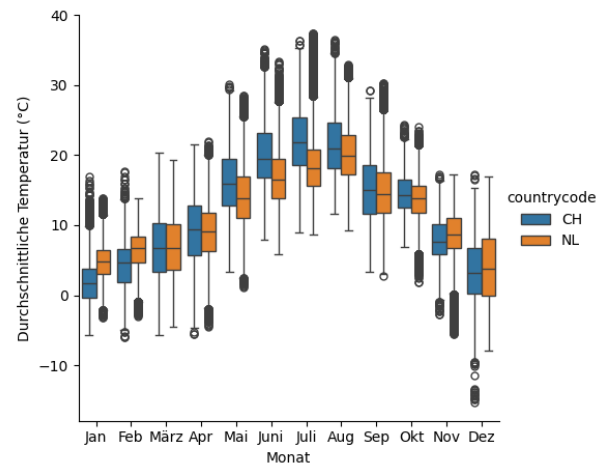


Zielbild an zwei Beispielen

Wetteranalyse in Städten unterschiedlicher Länder



Durchschnittliche monatliche Temperaturen für ausgewählte Städte



Analyse Literaturrecherche

ArXiv Abfragen

arXiv is a project by the Cornell University Library that provides open access to 1,000,000+ articles in Physics, Mathematics, Computer Science, Quantitative Biology, Quantitative Finance, and Statistics.

Wir benutzen [arXiv.org](https://arxiv.org) hier als Datenlieferantin. Es bietet ein stabiles API, wofür es einen einfach zu benutzenden Python-Wrapper gibt: arxiv.py. Das Ziel soll einerseits sein, eine sauber gegliederte Datenverarbeitungspipeline zu entwickeln, die das Resultat einer arxiv Abfrage in einen Pandas DataFrame abfüllt, und andererseits sollen Funktionen für explorative Analysen und Visualisierungen bereitgestellt werden.

Lernziele

Die Studierenden können

- komplexe Daten via API aus dem Internet zu beziehen.
- komplexe Daten in generische Datentypen (`dict`, `list`, `str`, `int`, `float`, `datetime`) zu transformieren.
- solche Daten nach gewissen Kriterien filtern und in einen [Pandas DataFrame](#) abzufüllen.
- das Package [Pipe](#) nutzen, um die Datenverarbeitung in einen sauberen Fluss zu bringen.
- explorative Analysen mithilfe von [Pandas](#) durchführen.
- statistische Visualisierungen der Daten mit [Seaborn](#) erstellen.

Voraussetzungen

Folgende Packages müssen mindestens installiert sein:

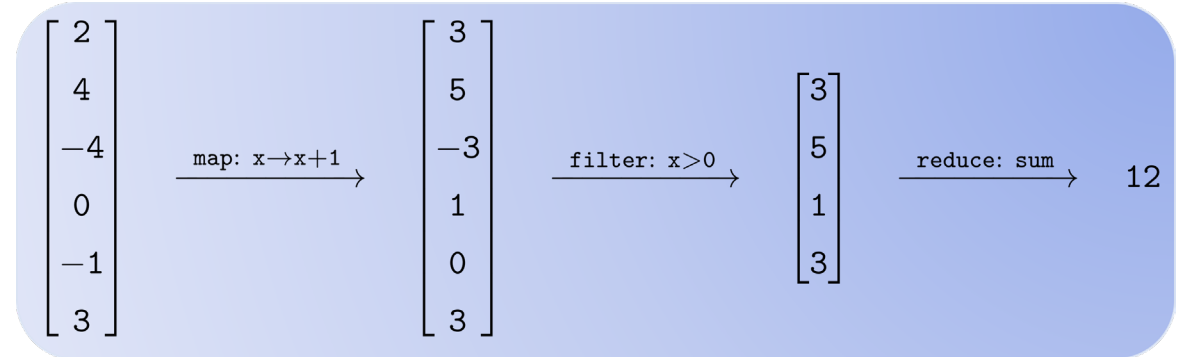
```
pip install arxiv pandas seaborn pipe
```

Eine erste arXiv Abfrage

In der Dokumentation von [arxiv](#) wird unter "Usage" beschrieben, wie Abfragen mithilfe eines `arxiv.Search` Objekts getätigt werden:

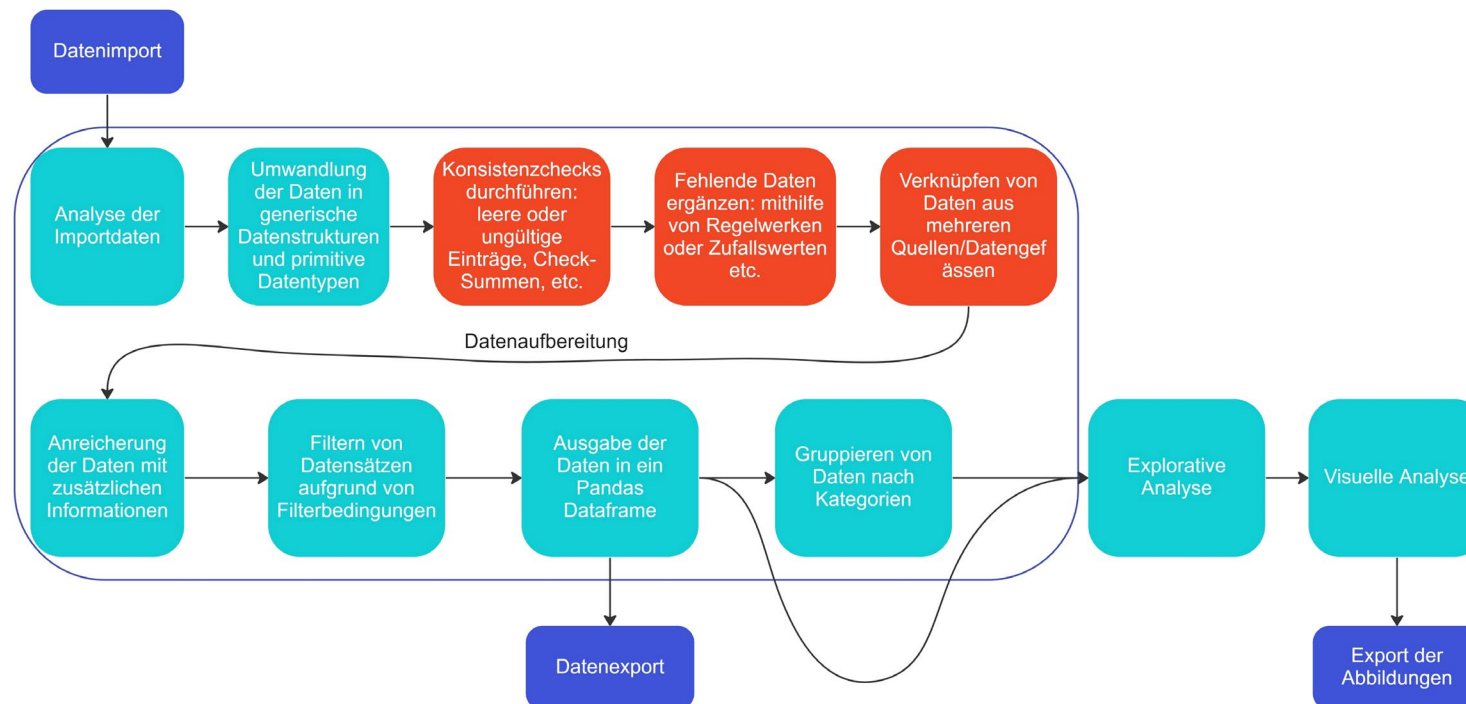
Inhalt

- Einführung in das Modul
 - Vorstellung
 - Lernziele
 - Einordnung in der Modullandschaft
 - Themenlandschaft / Semesterplan
 - Didaktisches Konzept / Arbeitsweise
 - Leistungsnachweis/Projektarbeit
 - Material und Arbeitsumgebung
 - Zielbild
- **Datenzentrierte Programmierung**
 - Grundprinzipien der Datenzentrierte Programmierung
- Repetition List-Comprehensions (List/Dict)
 - Live-Coding
- Selbststudium



Datenzentrierte Programmierung

- Das Ziel der datenzentrierten Programmierung ist eine scriptbasierte Pipeline aufzubauen, mit welcher Daten aus einer oder mehreren Quellen bezogen, aufbereitet, visualisiert und in geeigneter Form exportiert werden.
- Es sollen dabei moderne Datenstrukturen, funktionale Programmieransätze und explorative Visualisierungen der Daten zum Einsatz kommen.



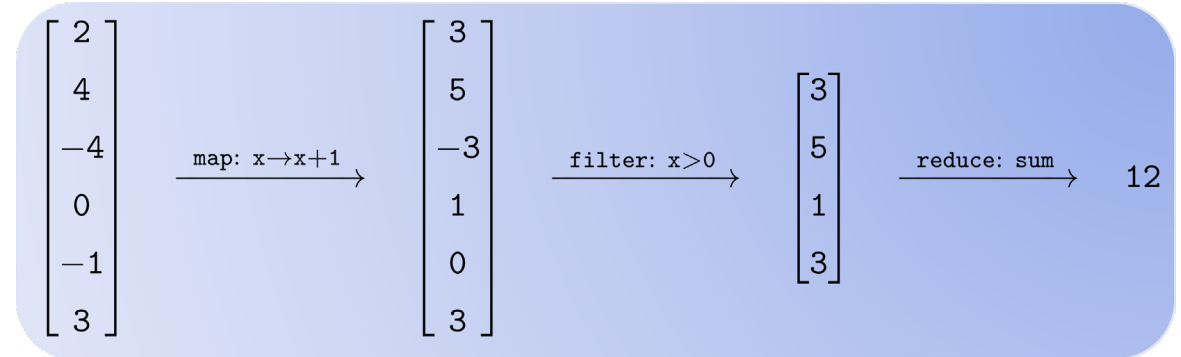
Mögliche Tasks
im Script an
einem Beispiel

Grundprinzipien der Datenzentrierten Programmierung

- Trennung der Zuständigkeiten (Separation of concerns)
 - + Nachvollziehbarer Code
 - + Einfache Lesbarkeit des Codes
- Seiteneffekte verhindern und isolieren (Avoid and isolate side effects)
 - + Einfacher zu parallelisieren
- Verwenden von generischen Datenstrukturen und primitiven Datentypen
 - + Direkte Verwendung in verschiedenen Packages, wie Pandas, Matplotlib usw.
- Transformation der Mutation vorziehen (Prefer transformation to mutation)
 - + Einfacher nachzuvollziehen
 - + Einfacher zu testen
 - + Einfacher zu parallelisieren
 - - Kann langsamer sein
- Unveränderbare Datenstrukturen vorziehen (Prefer immutable data structures)
 - + Klare Kommunikation der Nutzungsabsicht
 - + Sicher vor ungewollten Änderungen
 - - Langsamer und speicherintensiver
- Datenstrukturen (Modell) und Funktionalität (Verhalten) trennen
 - + Einfacher nachzuvollziehen
 - + Macht Code häufig allgemeiner und wiederverwendbarer
 - + Erhöht die Transparenz des Gesamtsystems

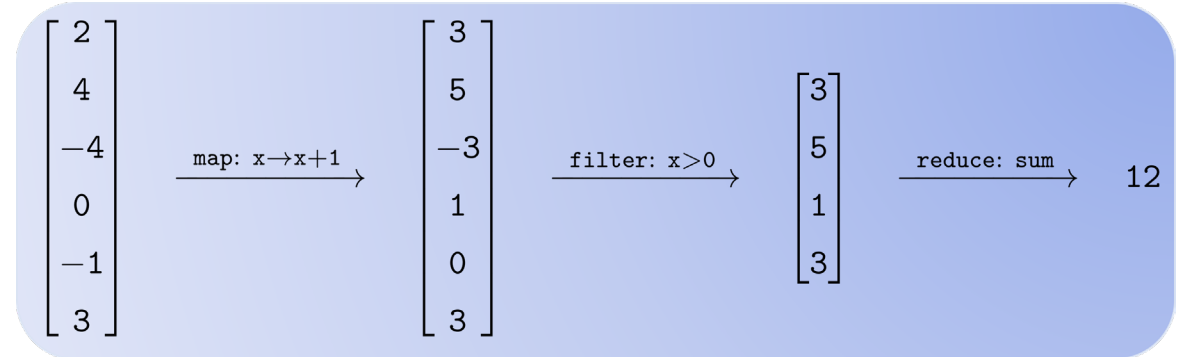
Inhalt

- Einführung in das Modul
 - Vorstellung
 - Lernziele
 - Einordnung in der Modullandschaft
 - Themenlandschaft / Semesterplan
 - Didaktisches Konzept / Arbeitsweise
 - Leistungsnachweis/Projektarbeit
 - Material und Arbeitsumgebung
 - Zielbild
- Datenzentrierte Programmierung
 - Grundprinzipien der Datenzentrierte Programmierung
- **Repetition List-Comprehensions (List/Dict)**
 - Live-Coding
- Selbststudium



Inhalt

- Einführung in das Modul
 - Vorstellung
 - Lernziele
 - Einordnung in der Modullandschaft
 - Themenlandschaft / Semesterplan
 - Didaktisches Konzept / Arbeitsweise
 - Leistungsnachweis/Projektarbeit
 - Material und Arbeitsumgebung
 - Zielbild
- Datenzentrierte Programmierung
 - Grundprinzipien der Datenzentrierte Programmierung
- Repetition List-Comprehensions (List/Dict)
 - Live-Coding
- **Selbststudium**



Selbststudium

- Starten mit Übungsblatt List-Comprehension
- Ziel Übungsblatt bis Ende Übungsstunde Mittwoch, 25. Februar fertiggestellt